Master Tracking System V3

Purpose

This is the new **final base system** for a persistent text simulation. It combines complete character tracking, world tracking, event continuity, NPC memory, session management, and canon database rules into one structure.[file:145][file:146][file:149][file:151]

The main design rule is simple: **the database is canon, the story text is presentation**. Narrative can describe, dramatize, and interpret events, but it may not change facts unless those changes are written back into the system state.[file:146][file:151]

Core Rules

Rule 1: Canon First

- The database is always the truth.[cite:151]
- Story text is never the source of truth.[cite:151]
- If story text and state conflict, the state wins.[cite:151]

Rule 2: Every Important Change Is Logged

- Stat growth must have a reason.[cite:146][cite:151]
- Promotions, injuries, betrayals, travel, battles, and discoveries must become events.[cite:145][cite:146][cite:151]
- NPC opinions and memories update after conversations, battles, major events, and time skips.[cite:145][cite:146]

Rule 3: Retrieval Over Raw History

- Never send the entire history into the AI every turn.[cite:146]
- Send only the current state, the relevant recent events, the most important long-term memories, and the current unresolved tensions.[cite:146][cite:145]

Rule 4: Validation Before Save

- No character can be in two places at once.[cite:151]
- No dead or incapacitated character should appear as active without recovery or override flags.[cite:151]
- Stats, loyalty values, relationship strength, and other bounded numbers must stay within defined ranges.[cite:151]

System Layers

1. Canon Database

The canon database is the permanent state layer. It stores characters, world state, factions, locations, events, session summaries, and validation metadata.[cite:146][cite:149][cite:151]

2. Character Tracking

This layer tracks who each person is, what they can do, what they want, what they remember, and how they change over time.[cite:145][cite:146]

3. World Tracking

This layer stores the current political, military, economic, seasonal, and geographic truth of the setting, including clan power balances and internal risks such as low-loyalty sub-factions.[cite:149]

4. Event Tracking

This layer stores all meaningful events in chronological form, including type, date, participants, location, consequences, and tags.[cite:146][cite:151]

5. Memory System

This layer stores what each NPC knows, suspects, believes, misremembers, values, and personally remembers about the player and the world.[cite:145][cite:146]

6. Session System

This layer creates a clean gameplay loop with opening snapshot, current objective, recent developments, end-of-session summary, and next-session setup.[cite:146]

7. Validation Layer

This layer checks contradictions before output or save and keeps the simulation internally consistent.[cite:151]

Top-Level Schema

```
{
  "meta": {
    "version": "3.0",
    "setting": "Japan 1540 - Sengoku",
    "canon_rule": "Database is truth; narrative is presentation only",
    "last_saved": "ISO_DATETIME"
  },
  "player_character_id": "li_hattori",
  "characters": {},
```

```
  "factions": {},
  "locations": {},
  "world_state": {},
  "events": [],
  "session_summaries": [],
  "prompt_cache": {},
  "validation_rules": {}
}
```

This keeps the structure simple enough to run in JSON while still supporting the larger systems already present in the current Python backend and tracking files.[cite:146][cite:151]

Character Tracking Schema

Each character should contain the following sections.

```
{
  "id": "li_hattori",
  "basic_info": {
    "name": "Li Hattori",
    "age": 17,
    "birthday": null,
    "clan": "Anegakoji",
    "stam": "Main House",
    "position": "Gokenin",
    "titles": ["Twin Demon's Son"],
    "location": "Anegakoji Mountain Stronghold",
    "availability": "active",
    "schedule": {
      "morning": "training",
      "afternoon": "command duties",
      "night": "private practice"
    }
  },
  "stats": {
    "LEA": 28,
    "VAL": 48,
    "INT": 23,
    "POL": 17,
    "rank": "C"
  },
  "skills": {
    "Single Sword": 65,
    "Two-Sword Mastery": 38,
    "Spear": 35,
    "Mountain Movement": 50,
    "Leadership Mastery": 30
  },
  "abilities": {
    "Twin Demon Flow": {
      "mastery": 12,
      "type": "combat",
      "unlock_condition": "extreme combat stress",
      "cooldown": "once per battle",
```

```
      "cost": "high stamina",
      "active": false
    }
  },
  "status": {
    "health": 82,
    "wounds": ["shoulder cut"],
    "fatigue": 44,
    "mental_state": "Driven",
    "morale": 91,
    "loyalty_to_clan": 93,
    "alive": true
  },
  "resources": {
    "money": 0,
    "troops_commanded": 200,
    "equipment": ["father_short_sword", "light_armor"],
    "supplies": {
      "food_days": 10,
      "arrows": 30
    }
  },
  "personality": {
    "core_traits": ["determined", "loyal", "ambitious"],
    "motivations": ["protect family", "rise in rank"],
    "fears": ["family dying", "failing father's legacy"],
    "goals": ["stop Mitsui betrayal", "survive coming war"],
    "secrets": ["secret two-sword style"]
  },
  "relationships": {
    "shiina_yasutane": {
      "type": "mentor",
      "strength": 88,
      "respect": 92,
      "trust": 85,
      "fear": 5,
      "notes": "Protector of family"
    }
  },
  "memory": {
    "player_interactions": [],
    "beliefs": [],
    "suspicions": [],
    "known_secrets": [],
    "misinformation": []
  },
  "history": {
    "key_events": [],
    "battles": [],
    "decisions": [],
    "promotions": [],
    "injuries": [],
    "stat_changes": []
  },
  "reputation": {
    "public": "rising hero",
```

```
    "clan_internal": "promising but politically risky",
    "enemy_view": "dangerous emerging commander"
  },
  "active_plots": []
}
```

This structure keeps the strong parts of the current character system—stats, skills, abilities, relationships, personality, memory, and current status—while making them more explicit and machine-readable.[cite:145][cite:146][cite:151]

Skills vs Abilities Rules

Keep the same distinction already used in the current system:

Type	Who has it	Range	Usage	Growth
Skills	Everyone	0-100	Always available	Training, repetition, combat use [cite:146]
Abilities	Rare or trained characters	0-100	Conditional	Extreme situations, special training, inheritance [cite:146]

Abilities should always include at least these extra fields:

- `unlock_condition`
- `cooldown`
- `cost`
- `trigger`
- `active`
- `last_used_event_id`

That preserves the clarity already present in the old tracking file while making the data easier to enforce in code.[cite:146]

Relationship System

Use one relationship object per target character, with multiple dimensions instead of a single number whenever needed.

```
{
  "target_id": "mitsui_sadamune",
  "type": "political_tension",
  "strength": -35,
  "respect": 40,
  "trust": 15,
  "fear": 25,
  "loyalty": 5,
  "rivalry": 45,
  "romance": 0,
```

```
  "family": 0,
  "last_updated": "Jaar 1540, Lente - 4e Maand, 8e Dag",
  "history": []
}
```

The existing system already tracks loyalty, rivalry, friendship, romantic ties, family ties, and bidirectional links, so this version simply makes the design more granular.[cite:145][cite:151]

NPC Memory Model

Every important NPC should have a dedicated memory block.

Memory Categories

- **Observed facts**: what they directly saw.[cite:145]
- **Heard rumors**: what they were told by others.[cite:145][cite:146]
- **Beliefs**: what they think is true, even if wrong.[cite:145]
- **Suspicions**: what they suspect but cannot prove.[cite:145]
- **Emotional marks**: grudges, gratitude, fear, admiration.[cite:145]
- **Player interaction memories**: direct moments with the player, each with date and effect.[cite:145]

Memory Record Template

```
{
  "memory_id": "mem_001",
  "date": "Jaar 1540, Lente - 3e Maand, 11e Dag",
  "type": "player_interaction",
  "summary": "Li warned indirectly about Mitsui activity",
  "certainty": 70,
  "emotional_weight": 85,
  "source": "direct",
  "tags": ["li_hattori", "mitsui", "trust"],
  "effect": {
    "trust_change": 5,
    "suspicion_change": 10
  }
}
```

This is the missing piece that turns a normal stat tracker into an actually persistent social simulation.[cite:145][cite:146]

World State Schema

The world file should become structured canon instead of only prose description.

```
{
  "current_date": {
    "year": 1540,
    "season": "Lente",
    "month": 5,
    "day": 12
  },
  "weather": "Clear",
  "economic_status": "Strained",
  "active_wars": ["Takeda vs Uesugi"],
  "major_tensions": [
    "Mitsui autonomy risk",
    "Takeda pressure",
    "Imagawa expansion"
  ],
  "global_flags": {
    "mitsui_rebellion_possible": true,
    "imagawa_searching_hattori_legacy": false
  },
  "power_balance": {
    "Imagawa": {"troops": 18000, "status": "strongest"},
    "Takeda": {"troops": 15000, "status": "expanding rapidly"},
    "Uesugi": {"troops": 12000, "status": "internal struggles"},
    "Shimazu": {"troops": 9000, "status": "southern power"},
    "Oda": {"troops": 8000, "status": "under pressure"},
    "Anegakoji": {"troops": 380, "status": "survival mode"}
  }
}
```

This draws directly from the current worldbuilding file, which already defines troop counts, internal clan risks, strategic position, and major faction tensions.[cite:149]

Faction Schema

Each clan should have its own record.

```
{
  "id": "anegakoji",
  "leader": "anegakoji_yoritsuna",
  "capital": "Mountain Stronghold",
  "troops": 380,
  "economy_koku": 45000,
  "specialties": ["mountain archery", "fortress defense", "guerrilla warfare"],
  "subfactions": {
    "main_house": {"loyalty": 95},
    "ema": {"loyalty": 88},
    "mitsui": {"loyalty": 75},
    "shiina": {"loyalty": 92},
    "kanamori": {"loyalty": 85}
```

```
  },
  "alliances": ["anti_takeda_sentiment"],
  "active_goals": ["survive regional wars", "contain Mitsui ambitions"],
  "risk_flags": ["internal betrayal risk"]
}
```

This preserves the internal stam politics and loyalty dynamics already described in the current world system.[cite:149][cite:145]

Event Schema

Every important action should be saved as an event.

```
{
  "event_id": 104,
  "date": "Jaar 1540, Lente - 5e Maand, 12e Dag",
  "event_type": "battle",
  "title": "Western Flank Held at Kawanakajima",
  "description": "Li Hattori and Yuki held the western chokepoint and prevented colla
  "location": "Kawanakajima",
  "participants": ["li_hattori", "yuki", "takeda_cavalry_detachment_1"],
  "consequences": {
    "li_hattori_reputation": "+major",
    "anegakoji_morale": "+12",
    "takeda_pressure": "+5",
    "western_flank_collapsed": false
  },
  "importance": "critical",
  "tags": ["battle", "reputation", "promotion", "western_flank"],
  "visibility": "public"
}
```

Your current Python backend already supports event IDs, participants, location, consequences, and importance, so this is a natural extension rather than a rewrite.[cite:151]

Session System

Session Opening Block

At the start of a session, generate a compact snapshot like this:

```
□ Jaar 1540, Lente - 5e Maand, 12e Dag
□ Anegakoji Mountain Stronghold
□ Li Hattori: Gokenin, commanding 200 troops
□ Current Objective: Stabilize western defenses and investigate Mitsui
⚠ Immediate Tensions: Takeda regrouping, Mitsui discontent, promotion politics
```

This follows the same structure as the current session-management format but makes it standardized for every run.[cite:146]

Session Ending Block

At the end of each session, store:

- important actions taken
- consequences
- changed stats
- changed loyalties
- unresolved threats
- next likely scene

This makes long-running continuity much stronger and easier to retrieve later.[cite:146]

Prompt Assembly System

For each turn, build the AI prompt from these sections only:

1. **System rules**
2. **Current player sheet**
3. **Relevant NPC sheets**
4. **Current world snapshot**
5. **Last 3 to 8 major events**
6. **Relevant long-term memories**
7. **Current objective and open tensions**
8. **User input**

Never dump the entire database into the prompt. The current files already imply session summarization and selective information display, so this just formalizes that logic.[cite:146] [cite:145]

Turn Loop

Use this exact loop:

1. Load database.
2. Validate current state.
3. Build prompt context from retrieval layers.
4. Generate narrative scene.
5. Extract state changes.
6. Apply state changes to canon data.
7. Log new events.
8. Update NPC memories.

9. Recalculate summaries.
10. Validate again.
11. Save database.

This is the single most important structural upgrade, because it prevents the AI from improvising canon without writing it back into state.[cite:151][cite:146]

Automatic Update Rules

After Every Conversation

- Update relationship values where relevant.[cite:145]
- Create NPC memory records for important lines, promises, insults, confessions, and discoveries.[cite:145]
- Update suspicion, trust, or respect if the interaction meaningfully changed them.[cite:145]

After Every Battle

- Log battle event.[cite:145][cite:151]
- Apply wounds, deaths, fatigue, morale shifts, and respect changes.[cite:145]
- Update troop strength and supplies when needed.[cite:149]

After Every Time Skip

- Advance date.[cite:146][cite:151]
- Update aging, projects, training gains, recovery, travel completion, and faction developments.[cite:145][cite:146]
- Trigger world-side changes even when the player did nothing directly.[cite:149]

After Every Major Event

- Update public reputation and hidden political consequences.[cite:145]
- Reveal or generate follow-up risks, such as rebellion, promotion resistance, assassination plots, or external invasion pressure.[cite:149][cite:145]

Validation Rules

Use these checks before every save.

Character Validation

- Age must be realistic or intentionally flagged.[cite:151]
- Stats must stay within 0 to 100.[cite:151]
- Rank must match stat logic or include an override reason.[cite:151]
- A dead character cannot be marked available unless a supernatural or mistaken-report flag exists.
- Troops commanded cannot exceed plausible role limits unless promoted.

Relationship Validation

- Relationship strength stays in range.[cite:151]
- If bidirectional mode is enabled, reverse links must exist.[cite:151]
- A “family” relation cannot appear between unrelated IDs without explicit data.

World Validation

- A location cannot be controlled by mutually exclusive factions at the same time unless contested.
- A clan cannot have both “destroyed” and “stable economy” flags simultaneously.
- Date only moves forward.[cite:151]

Event Validation

- All participants must exist or be defined as abstract groups.[cite:151]
- Critical events must have consequences.
- Public events should affect at least one reputation, morale, or faction flag.

Suggested Python Upgrade Plan

The current backend already has dataclasses, stat clamping, relationship tracking, event logging, save/load, and character-sheet generation.[cite:151] The best upgrade path is:

1. Add `memory`, `resources`, `reputation`, `active_plots`, and `schedule` fields to character records.[cite:145][cite:151]
2. Add structured `factions` and `locations` dictionaries beside `world_state`.[cite:149][cite:151]
3. Add a validation module before save and after load.[cite:151]
4. Add a prompt builder function that collects only relevant context.[cite:146]
5. Add event tags and visibility fields.[cite:151]
6. Add session-summary creation after major scenes.[cite:146]

Command Outputs

The system should continue supporting quick commands, but they should now be generated from canon data.

Command	Output
`Status`	Full character sheet [cite:146][cite:151]
`Skills`	Current skill and ability values [cite:146]
`World`	Political and economic snapshot [cite:146][cite:149]
`Time`	Exact current date and active developments [cite:146][cite:151]
`History`	Major chronological events [cite:146][cite:151]
`Relations`	Current relationship map [cite:145][cite:151]
`Threats`	Open risks and unresolved tensions [cite:149]

Final Standard

This system should be treated as the **master version**. The older files remain useful as source material, but this document becomes the main design reference because it unifies complete character tracking, world state, event continuity, NPC memory, skills versus abilities, session management, and database-backed canon rules into one format.[cite:145][cite:146][cite:149][cite:151]

The real strength of this version is not that it adds more lore, but that it makes the simulation structurally consistent. That is what prevents drift, forgotten details, broken continuity, and random tone changes over long sessions.[cite:145][cite:146][cite:151]